
Práctica 4

Arquitecturas

Paralelas

Universidad de Córdoba
Escuela Politécnica Superior

-

Grado de Ingeniería Informática

-

Arquitecturas Paralelas

★ Juan Manuel Ramírez Hinojosa (i22rahij)



Ejercicio Práctica IV

Teniendo en cuenta el código que figura en el último apartado del tema de “Multiprocesadores de Memoria Compartida” y que se muestra a continuación:

```
int *array;
int length, count;

int count3s()
{
    int i;
    count = 0;
    for (i=0; i<length; i++)
    {
        if (array[i] == 3)
            count++;
    }
    return count;
}
```

- A. Realizar la programación de dicho código, que cuenta el número de veces que aparece el número 3 en un vector, midiendo el tiempo de ejecución del mismo para un vector de números enteros de, al menos, 1 millón de elementos generados de forma aleatoria entre 0 y 100.
- B. Implementar una versión paralela de dicho código con Pthreads, incorporando sección crítica. Similar al código mostrado en el mencionado documento y que se indica a continuación, obteniendo también el tiempo de ejecución:

```
int t; // numero de threads
int *array, length, count;

int count3s()
{
    int i; count = 0;
    for (i=0; i<t; i++) // crear t threads
        thread_create(count3s_thread, i);
    return count;
}
```



```

mutex m;

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
        {
            mutex_lock(m);
            count++;
            mutex_unlock(m);
        }
}

```

- C. Realizar la implementación paralela mediante Pthreads contemplando la estructura “paddeada” para cada contador de hilo tal y como figura al final del citado documento y que se muestra a continuación, obteniendo los tiempos de ejecución.

```

int t;    // numero de threads
int *array, length, count;

int count3s()
{
    int i; count = 0;
    for (i=0; i<t; i++)    // crear t threads
        thread_create(count3s_thread, i);
    return count;
}

mutex m;
struct padded_int
{
    int value;
    char padding[60];
} private_count[MaxThreads];

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
            private_count[id].value++;
    mutex_lock(m);
    count += private_count[id].value;
    mutex_unlock(m);
}

```



- D. Para cada una de las implementaciones paralelas anteriores (b y c): Comparar las ejecuciones para distinto número de hilos (2, 4, 8, 16, 32), obteniendo el speed-up y realizando una gráfica de las mismas.
- E. Justificar los resultados obtenidos.
- F. Implementar, mediante OpenMP, una versión paralela del código desarrollado en el apartado a) para distinto número de hilos, obteniendo el speed-up. Desarrollo de conclusiones.

Nota: Para medir los tiempos tanto del programa secuencial como de los programas paralelos pthread, utilizar la función “omp_get_wtime()”. Recordar incluir la librería omp.h y compilar con el flag -fopenmp.



Apartado A

Se ha realizado el código para rellenar y contar los 3 de un vector cuyo tamaño se ha modificado a 1 mil millón para obtener resultados de tiempo más grandes y más fáciles de comparar. El código resultante sería:

```
#include <omp.h>
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

#define SIZE 1000000000

int count3s(int array[], int lenght)
{
    int i;
    int count = 0;
    for(i=0;i<lenght;i++)
    {
        if(array[i]==3)
        {
            count++;
        }
    }
    return count;
}

int main()
{
    int *array = malloc(SIZE * sizeof(int));
    if (array == NULL) {
        printf("Error: No se pudo asignar memoria\n");
        return 1;
    }
    int lenght = SIZE;
    double t0 = omp_get_wtime();
    srand(time(NULL));

    for(int i=0;i<lenght;i++)
    {
        array[i] = rand() % 101;
    }
}
```



```

int res=count3s(array,lenght);
double t1 = omp_get_wtime();
printf("Valor 100 del vector: %d\n",array[100]);
printf("Numero de 3 en el vector: %d\n", res);
printf("Tiempo transcurrido = %f segundos\n",t1-t0);
}

```

Con este código se ha realizado el proceso de compilación y ejecución asegurando el uso de la flag -fopenmp en la compilación para el correcto funcionamiento de la función omp_get_wtime() la cual nos permite medir el tiempo de ejecución como se muestra a continuación.

```

juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ gcc practica4.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 28
Numero de 3 en el vector: 9905384
Tiempo transcurrido = 10.430990 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 53
Numero de 3 en el vector: 9904186
Tiempo transcurrido = 9.916589 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 6
Numero de 3 en el vector: 9901246
Tiempo transcurrido = 9.883803 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 34
Numero de 3 en el vector: 9904477
Tiempo transcurrido = 10.280044 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 68
Numero de 3 en el vector: 9898322
Tiempo transcurrido = 9.894787 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 91
Numero de 3 en el vector: 9906854
Tiempo transcurrido = 10.050453 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$

```



Apartado B

Con la información aportada se ha realizado la modificación al código añadiendo sección crítica y empleando pthread para conseguir una ejecución paralela. Para esta primera prueba se ha decidido usar 4 hilos al paralelizar. Este sería el código:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define SIZE 1000000000
#define NUM_THREADS 4

int *array;
int total_count = 0;
pthread_mutex_t lock;

typedef struct {
    int start;
    int end;
} ThreadArgs;

void* count3s_parallel(void *args) {
    ThreadArgs *data = (ThreadArgs *)args;
    int local_count = 0;

    for (int i = data->start; i < data->end; i++) {
        if (array[i] == 3) {
            local_count++;
        }
    }

    pthread_mutex_lock(&lock);
    total_count += local_count;
    pthread_mutex_unlock(&lock);

    pthread_exit(NULL);
}

int main() {
    array = malloc(SIZE * sizeof(int));
```



```

    if (array == NULL) {
        printf("Error: No se pudo asignar memoria\n");
        return 1;
    }

    srand(time(NULL));
    for (int i = 0; i < SIZE; i++) {
        array[i] = rand() % 101;
    }

    double t0 = omp_get_wtime();

    pthread_t threads[NUM_THREADS];
    ThreadArgs args[NUM_THREADS];
    int chunk_size = SIZE / NUM_THREADS;

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        args[i].start = i * chunk_size;
        args[i].end = (i == NUM_THREADS - 1) ? SIZE : args[i].start +
chunk_size;
        pthread_create(&threads[i], NULL, count3s_parallel, (void
*) &args[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    double t1 = omp_get_wtime();

    printf("Valor 100 del vector: %d\n", array[100]);
    printf("Numero de 3 en el vector: %d\n", total_count);
    printf("Tiempo transcurrido = %f segundos\n", t1 - t0);

    free(array);
    return 0;
}

```



En base a este código se vuelve a realizar la compilación y ejecución, esta vez añadiendo la flag `-lpthread` para asegurar el correcto funcionamiento de los hilos del programa. Tras este proceso se han obtenido estos tiempos:

```
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ gcc practica4_pthread.c -fopenmp -lpthread
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 48
Numero de 3 en el vector: 9901384
Tiempo transcurrido = 0.307533 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 37
Numero de 3 en el vector: 9899702
Tiempo transcurrido = 0.308770 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 9
Numero de 3 en el vector: 9901480
Tiempo transcurrido = 0.307809 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 12
Numero de 3 en el vector: 9901483
Tiempo transcurrido = 0.398929 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 1
Numero de 3 en el vector: 9900478
Tiempo transcurrido = 0.386362 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 100
Numero de 3 en el vector: 9899901
Tiempo transcurrido = 0.632293 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ |
```

Como podemos ver, el programa paralelo es mucho más eficiente en cuanto a tiempo con respecto al programa secuencial como ya se demostró en la práctica anterior.



Apartado C

Para este apartado se ha decidido crear una estructura en la que almacenemos un entero de 4 B y una cadena de 60 caracteres como relleno para así ocupar una línea de caché de 64 B. Este sería el código modificado:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <omp.h>

#define SIZE 100000000
#define NUM_THREADS 4

int *array;
int length = SIZE;
int count = 0;

pthread_mutex_t m;

// Estructura "paddeada" para evitar false sharing
typedef struct {
    int value;
    char padding[60]; // para llenar una cache line (usualmente 64
bytes)
} padded_int;

padded_int private_count[NUM_THREADS];

typedef struct {
    int id;
} ThreadArg;

void* count3s_thread(void* arg) {
    ThreadArg* threadArg = (ThreadArg*) arg;
    int id = threadArg->id;

    int length_per_thread = length / NUM_THREADS;
    int start = id * length_per_thread;
    int end = (id == NUM_THREADS - 1) ? length : start +
length_per_thread;
```



```

        for (int i = start; i < end; i++) {
            if (array[i] == 3) {
                private_count[id].value++;
            }
        }

        pthread_mutex_lock(&m);
        count += private_count[id].value;
        pthread_mutex_unlock(&m);

        pthread_exit(NULL);
    }
}

int main() {
    array = malloc(SIZE * sizeof(int));
    if (array == NULL) {
        printf("Error: No se pudo asignar memoria\n");
        return 1;
    }

    // Inicializar vector aleatoriamente
    srand(time(NULL));
    for (int i = 0; i < SIZE; i++) {
        array[i] = rand() % 101;
    }

    pthread_t threads[NUM_THREADS];
    ThreadArg threadArgs[NUM_THREADS];
    pthread_mutex_init(&m, NULL);

    // Inicializar contadores privados
    for (int i = 0; i < NUM_THREADS; i++) {
        private_count[i].value = 0;
    }

    double t0 = omp_get_wtime();

    for (int i = 0; i < NUM_THREADS; i++) {
        threadArgs[i].id = i;
        pthread_create(&threads[i], NULL, count3s_thread,
(void*)&threadArgs[i]);
    }
}

```



```

for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(threads[i], NULL);
}

double t1 = omp_get_wtime();

printf("Valor 100 del vector: %d\n", array[100]);
printf("Numero de 3 en el vector: %d\n", count);
printf("Tiempo transcurrido = %f segundos\n", t1 - t0);

free(array);
pthread_mutex_destroy(&m);

return 0;
}

```

Este código se compila y ejecuta de la misma manera que en el anterior apartado y obtenemos los siguientes tiempos que, como se puede ver, han sido drásticamente reducidos con respecto a la anterior modificación debido al uso eficiente de caché.

```

juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ gcc practica4_padded.c -fopenmp -lpthread
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 54
Numero de 3 en el vector: 989237
Tiempo transcurrido = 0.036975 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 47
Numero de 3 en el vector: 989633
Tiempo transcurrido = 0.036914 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 32
Numero de 3 en el vector: 991297
Tiempo transcurrido = 0.039158 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 4
Numero de 3 en el vector: 990150
Tiempo transcurrido = 0.036131 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 24
Numero de 3 en el vector: 990642
Tiempo transcurrido = 0.039070 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ ./a.out
Valor 100 del vector: 78
Numero de 3 en el vector: 989996
Tiempo transcurrido = 0.037112 segundos
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p4$ |

```



Apartado D

Para realizar las gráficas se ha realizado el mismo procedimiento que en la práctica anterior; ejecutando cada programa con cada número de hilos 6 veces y recopilando los tiempos de ejecución.

Utilizando los datos del programa sin relleno se ha obtenido la siguiente tabla con su correspondiente gráfica.

	Paralela Sin Relleno					
Nº Hilos	1	2	4	8	16	32
Tiempo Medio	1240.02	630.62	390.28	292.02	156.07	159.90
Valor más alto	1287.25	690.53	632.29	321.34	160.92	164.41
Valor más bajo	1219.80	614.89	307.53	248.99	152.21	157.17
Variación	67.45	75.63	324.76	72.35	8.71	7.24
Speed-Up	1.00	1.97	3.18	4.25	7.95	7.75

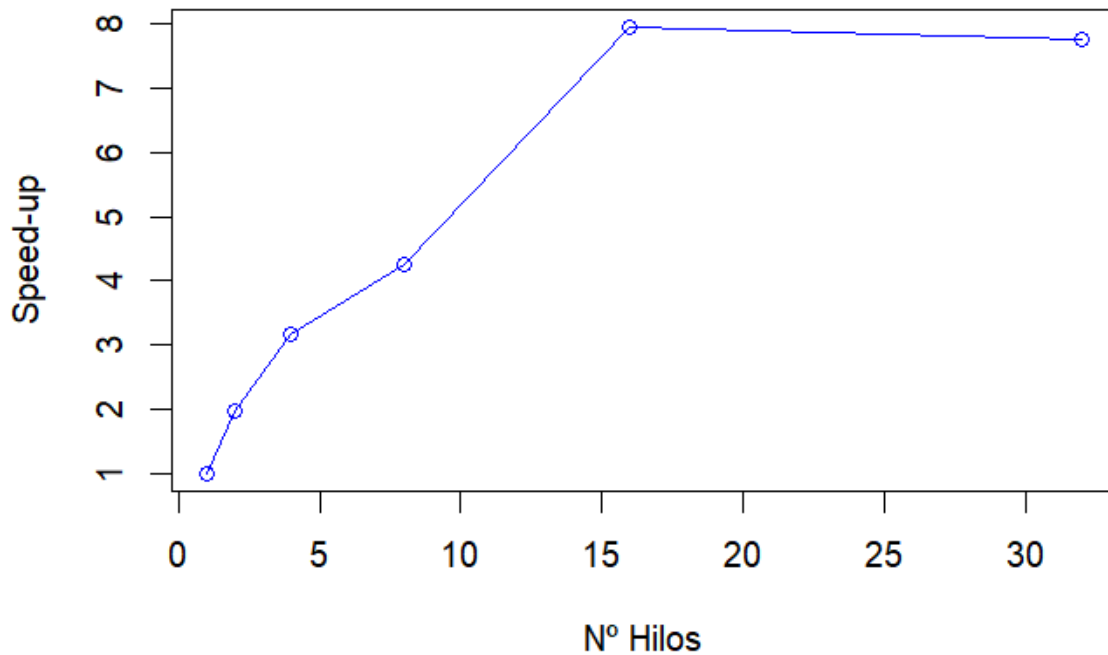
* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos

La curva de rendimiento resultante de estos datos presenta la siguiente forma:



Curva de rendimiento Sin Relleno



Por otra parte, los datos recogidos del programa paralelo con relleno son los mostrados en esta siguiente tabla.

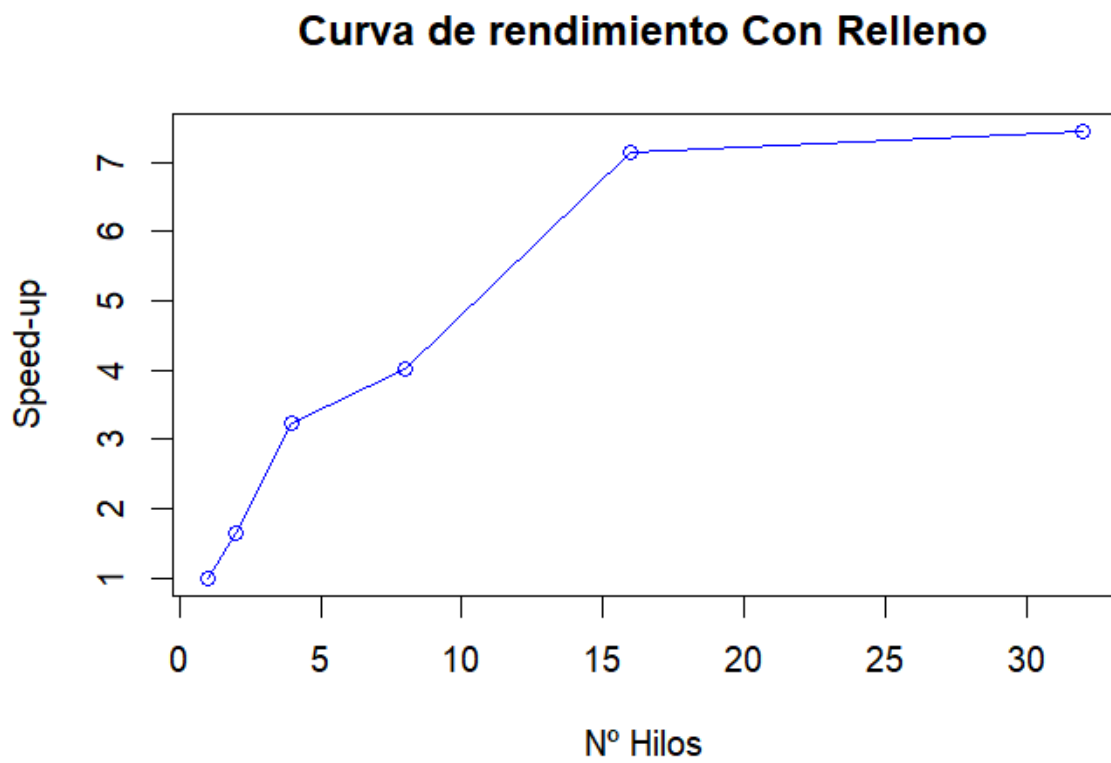
	Paralela Con Relleno					
N° Hilos	1	2	4	8	16	32
Tiempo Medio	1224.32	746.93	377.40	304.35	171.30	164.47
Valor más alto	1254.14	948.96	453.76	305.41	222.51	166.05
Valor más bajo	1216.67	697.78	349.71	303.31	155.97	160.97
Variación	37.47	251.18	104.05	2.1	66.54	5.08
Speed-Up	1.00	1.64	3.24	4.02	7.15	7.44

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos



Y su correspondiente gráfica con la curva de rendimiento:



Apartado E

El **tiempo medio de ejecución del programa secuencial** fue aproximadamente **10,08 segundos**, y se utilizó como referencia para calcular el *speed-up* de las versiones paralelas.

La **versión paralela sin relleno** mostró un aumento significativo del rendimiento hasta alcanzar un *speed-up* cercano a 8x con 16 hilos. Sin embargo, a partir de ese punto el rendimiento se estanca, debido principalmente a **problemas de contención en la sección crítica y false sharing**, lo cual limita su escalabilidad.

La **versión paralela con relleno**, aunque mostró un *speed-up* ligeramente menor inicialmente, evidenció un comportamiento **más estable y escalable** conforme se aumentaba el número de hilos. Gracias al uso de estructuras “paddeadas” por hilo, se reducen los conflictos de caché, mejorando el rendimiento en arquitecturas con muchos núcleos. Con todos los datos presentados, podemos concluir lo siguiente:

La **versión sin relleno es más rápida con pocos hilos**, pero se ve penalizada a medida que se incrementa la concurrencia.

La **versión con relleno ofrece un rendimiento más predecible y sostenible**, siendo más adecuada para sistemas multinúcleo.

En función de la arquitectura del sistema y el número de hilos disponibles, **el uso de relleno puede marcar la diferencia en aplicaciones altamente paralelas**.



Apartado F

El código modificado para el uso de OpenMP en la paralelización es el siguiente:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000000000

int count3s_parallel(int array[], int length, int num_threads) {
    int count = 0;

    #pragma omp parallel for reduction(+:count)
    for (int i = 0; i < length; i++) {
        if (array[i] == 3) {
            count++;
        }
    }

    return count;
}

int main() {
    int *array = malloc(SIZE * sizeof(int));
    if (array == NULL) {
        printf("Error: No se pudo asignar memoria\n");
        return 1;
    }

    srand(time(NULL));
    for (int i = 0; i < SIZE; i++) {
        array[i] = rand() % 101;
    }

    int num_threads[] = {1, 2, 4, 8, 16, 32}; // Puedes ajustar esta
    lista

    for (int i = 0; i < sizeof(num_threads)/sizeof(num_threads[0]);
    i++) {
        double t0 = omp_get_wtime();
```



```

    int result = count3s_parallel(array, SIZE, num_threads[i]);
    double t1 = omp_get_wtime();

    printf("\n--- Con %d hilos ---\n", num_threads[i]);
    printf("Valor 100 del vector: %d\n", array[100]);
    printf("Número de 3 en el vector: %d\n", result);
    printf("Tiempo transcurrido: %f segundos\n", t1 - t0);
}

free(array);
return 0;
}

```

Utilizando los datos del programa sin relleno se ha obtenido la siguiente tabla con su correspondiente gráfica.

	Paralela OpenMP					
Nº Hilos	1	2	4	8	16	32
Tiempo Medio	1284.77	563.61	303.24	195.83	131.75	136.39
Valor más alto	1697.99	586.12	360.62	218.16	138.99	143.44
Valor más bajo	1118.23	556.61	282.77	183.77	126.01	132.93
Variación	579.76	29.51	77.85	34.39	12.98	10.51
Speed-Up	1.00	2.28	4.24	6.56	9.75	9.42

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos

La curva de rendimiento resultante de estos datos presenta la siguiente forma:



Con todos estos datos podemos llegar a las siguientes conclusiones:

La paralelización mediante OpenMP **mejoró significativamente el tiempo de ejecución** respecto a la versión secuencial, especialmente con un número moderado de hilos (hasta 16).

El **mayor Speed-Up** se alcanzó con 16 hilos, logrando una aceleración de casi 10 veces respecto al caso secuencial.

A partir de 16 hilos, el rendimiento comenzó a **saturarse**, e incluso empeoró ligeramente con 32 hilos. Esto se debe al **overhead en la gestión de hilos** y a las **limitaciones físicas del hardware**, como el número de núcleos reales disponibles.

La **variabilidad en los tiempos de ejecución disminuyó** a medida que aumentaba el número de hilos, lo cual indica una ejecución más estable en entornos altamente paralelos.

En resumen, el uso de OpenMP demuestra ser una solución eficiente y sencilla para aprovechar la concurrencia, pero es fundamental **ajustar el número de hilos** en función de la arquitectura del sistema para obtener el máximo rendimiento sin incurrir en sobrecostes de paralelización.

